

Batch: H5 Roll No.: 16010122158

Experiment 01

Title: Data pre-processing using python on real world dataset

Objective:

1. To learn how to prepare the dataset
2. To learn various steps in Data –Preprocessing
3. Perform the Descriptive analytics (obtain measure of central tendency, dispersion, skewness, Kurtosis, and graphical representation of the given dataset)
4. Perform Exploratory Data Analysis over dataset

Course Outcome:

CO1: Understand basic concepts of data analytics to solve real-world problems

CO2: Experiment using advanced software techniques and tools to conduct thorough and insightful analysis

Books/ Journals/ Websites referred:

1. Data Mining Concepts and Techniques Jiawei Han, Michelin Kamber, Jian Pie, 3rd edition

Resources used:

1. Kaggle datasets
2. Google Colaboratory

Theory (About Data Preprocessing):

Data preprocessing involves transforming raw data to well-formed data sets so that data mining analytics can be applied. Raw data is often incomplete and has inconsistent formatting. The adequacy or inadequacy of data preparation has a direct correlation with the success of any project that involve data analytics.

Preprocessing involves both data validation and data imputation. The goal of data validation is to assess whether the data in question is both complete and accurate. The goal of data imputation is to correct errors and input missing values – either manually or automatically through business process automation (BPA) programming.

Source of the dataset (URL):

<https://www.kaggle.com/datasets/jessemostipak/hotel-booking-demand>

```
# Loading data
df = pd.read_csv('/content/drive/My Drive/hotel_bookings.csv')
df.head()
```

	hotel	is_canceled	lead_time	arrival_date_year	arrival_date_month	arrival_date_week_number	arrival_date_day_of_month	stays_in_weekend_nights	stays_in_week_nights	adults	...
0	Resort Hotel	0	342	2015	July	27	1	0	0	2	...
1	Resort Hotel	0	737	2015	July	27	1	0	0	2	...
2	Resort Hotel	0	7	2015	July	27	1	0	1	1	...
3	Resort Hotel	0	13	2015	July	27	1	0	1	1	...
4	Resort Hotel	0	14	2015	July	27	1	0	2	2	...

5 rows × 32 columns

Platform used by the student: Google Colaboratory (Python)

Following points should be written by students

Different steps in Data Preprocessing:

- Finding missing, null values
- Replacing missing, null values with statistical parameters
- Encoding categorical data if needed (Write user defined function)
- Normalization (Write user defined function)
- Discretization (Write user defined function)

Working (Paste the code and Output for each Data Preprocessing task):

Data Preprocessing:

1. Finding missing/null values:

```
df.isnull().sum()

hotel      0
is_canceled 0
lead_time  0
arrival_date_year  0
arrival_date_month  0
arrival_date_week_number  0
arrival_date_day_of_month  0
stays_in_weekend_nights  0
stays_in_week_nights  0
adults      0
children    4
babies      0
meal        0
country     488
market_segment  0
distribution_channel  0
is_repeated_guest  0
previous_cancellations  0
previous_bookings_not_canceled  0
reserved_room_type  0
assigned_room_type  0
booking_changes  0
deposit_type  0
agent       16340
company     112593
days_in_waiting_list  0
customer_type  0
adr         0
required_car_parking_spaces  0
total_of_special_requests  0
reservation_status  0
reservation_status_date  0
dtype: int64
```

2. Replacing missing/null values with statistical parameters:

The 'children', 'country', 'agent' and 'company' columns contain missing/null values.

Dropping 'agent' and 'company' columns as they have too many missing values.

Replacing missing values in the 'country' column with the most frequent country.

Missing values in 'children' column are replaced with 0.

```
✓ [4] df = df.drop(columns=['agent', 'company'])
```

```
✓ [5] most_frequent_country = df['country'].mode()[0]
      df['country'] = df['country'].fillna(most_frequent_country)
```

```
✓ [7] df['children'] = df['children'].fillna(0)
```

```
✓ df.isnull().sum()
```

```
hotel 0
is_canceled 0
lead_time 0
arrival_date_year 0
arrival_date_month 0
arrival_date_week_number 0
arrival_date_day_of_month 0
stays_in_weekend_nights 0
stays_in_week_nights 0
adults 0
children 0
babies 0
meal 0
country 0
market_segment 0
distribution_channel 0
is_repeated_guest 0
previous_cancellations 0
previous_bookings_not_canceled 0
reserved_room_type 0
assigned_room_type 0
booking_changes 0
deposit_type 0
days_in_waiting_list 0
customer_type 0
adr 0
required_car_parking_spaces 0
total_of_special_requests 0
reservation_status 0
reservation_status_date 0
dtype: int64
```

All missing/null values are dealt with.

3. Encoding categorical data:

```

df.dtypes

hotel                object
is_canceled          int64
lead_time            int64
arrival_date_year     int64
arrival_date_month   object
arrival_date_week_number int64
arrival_date_day_of_month int64
stays_in_weekend_nights int64
stays_in_week_nights int64
adults               int64
children             float64
babies               int64
meal                 object
country              object
market_segment       object
distribution_channel  object
is_repeated_guest    int64
previous_cancellations int64
previous_bookings_not_canceled int64
reserved_room_type   object
assigned_room_type   object
booking_changes       int64
deposit_type          object
days_in_waiting_list int64
customer_type         object
adr                  float64
required_car_parking_spaces int64
total_of_special_requests int64
reservation_status    object
reservation_status_date object
dtype: object

```

The 'arrival_date_month' is one of the columns containing categorical data.

Replacing month names by respective numbers 1-12.

```
✓ [10] encoding = {"arrival_date_month": {"January": 1, "February": 2, "March": 3, "April": 4, "May": 5,
0s "June": 6, "July": 7, "August": 8, "September": 9,
"October": 10, "November": 11, "December": 12}}

df = df.replace(encoding)
```

```
✓ df.head()
0s
```

	hotel	is_canceled	lead_time	arrival_date_year	arrival_date_month	arrival_date_week_number	arrival_date_day_of_month	stays_in_weekend_nights	stays_in_week_nights	adults	...
0	Resort Hotel	0	342	2015	7	27	1	0	0	2	...
1	Resort Hotel	0	737	2015	7	27	1	0	0	2	...
2	Resort Hotel	0	7	2015	7	27	1	0	1	1	...
3	Resort Hotel	0	13	2015	7	27	1	0	1	1	...
4	Resort Hotel	0	14	2015	7	27	1	0	2	2	...

5 rows × 30 columns

4. Removing incorrect data:

Removing any 0 values from adults column as bookings need to have at least one adult.

```
✓ [13] df = df[df['adults'] != 0]
0s (df['adults'] == 0).any()

False
```

5. Discretization:

Making a seasons column and mapping arrival month to the season.

```

✓ [24] seasons = {
0s      'Summer': ['February', 'March', 'April', 'May'],
      'Monsoon': ['June', 'July', 'August', 'September'],
      'Winter': ['October', 'November', 'December', 'January', 'February']}

def month_to_season(month):
    for season, months in seasons.items():
        if month in months:
            return season

df['season'] = df['arrival_date_month'].apply(month_to_season)
  
```

```

✓ [25] df.head()
0s

```

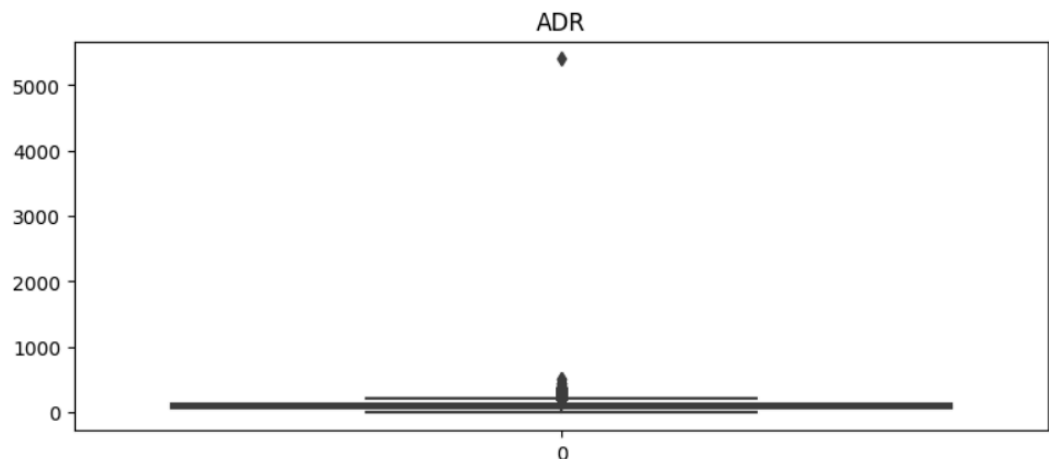
	booking_changes	deposit_type	days_in_waiting_list	customer_type	adr	required_car_parking_spaces	total_of_special_requests	reservation_status	reservation_status_date	season
...	3	No Deposit	0	Transient	0.0	0	0	Check-Out	2015-07-01	Monsoon
...	4	No Deposit	0	Transient	0.0	0	0	Check-Out	2015-07-01	Monsoon
...	0	No Deposit	0	Transient	75.0	0	0	Check-Out	2015-07-02	Monsoon
...	0	No Deposit	0	Transient	75.0	0	0	Check-Out	2015-07-02	Monsoon
...	0	No Deposit	0	Transient	98.0	0	1	Check-Out	2015-07-03	Monsoon

6. Removing outliers:

```

✓ [15] # Checking for outliers in ADR column
0s

plt.figure(figsize=(20,30))
plt.subplot(7, 2, 1)
sns.boxplot(df['adr'])
plt.title('ADR')
plt.show()
  
```

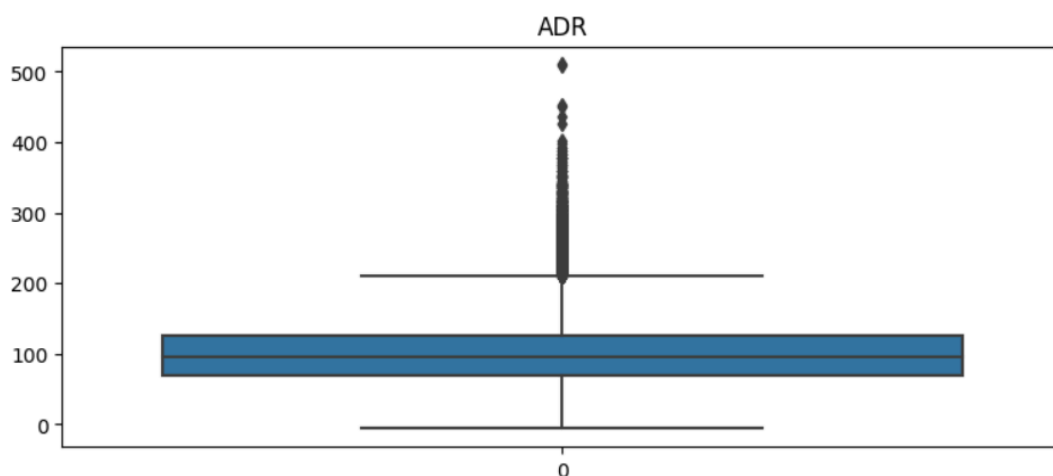


```

✓ [33] # Removing outliers
0s df.drop(df[df['adr'] > 5000].index, inplace = True)
  
```

```

✓ [34] plt.figure(figsize=(20,30))
0s plt.subplot(7, 2, 1)
    sns.boxplot(df['adr'])
    plt.title('ADR')
    plt.show()
  
```



Conclusion (Students should write in their own words):

Prepared the dataset and pre-processed it using python. Handled missing/null values, incorrect data, duplicate data, encoded categorical variables and removed outliers.

Performed descriptive analysis on the dataset and drew some conclusions using exploratory data analysis.

Post lab questions:

Q.1 What are some common challenges encountered during data cleaning? How did you handle missing values in the provided dataset?

Challenges in Data Cleaning:

Missing Values: One of the most common issues, missing data can occur due to various reasons, such as human error, equipment malfunction, or data corruption. Handling missing values is crucial to ensure the quality of the analysis.

- **Duplicate Entries:** Duplicate records can skew the results and need to be identified and removed.
- **Incorrect Data:** Erroneous data, such as outliers or inconsistent values, can affect the analysis and need to be corrected or removed.
- **Inconsistent Data Formats:** Data coming from different sources may have inconsistent formats (e.g., date formats), requiring standardization.
- **Categorical Data:** Converting categorical variables into numerical formats suitable for machine learning models can be challenging.

Handling Missing Values: In the provided dataset, missing values were handled by first identifying rows containing 'null' values. These rows were then removed to ensure that the dataset only contained complete entries. Specifically, I applied a lambda function that checks each row for the presence of 'null' and filtered out any row where 'null' was detected. This method ensured that the remaining dataset was free of missing values, which is essential for accurate analysis.

Q.2 Explain the importance of data normalization in the context of machine learning models. How does normalizing benefit the analysis?

Importance of Data Normalization: Data normalization is the process of scaling numerical data to a standard range, usually between 0 and 1 or -1 and 1. It's particularly important in machine learning for several reasons:

- **Improved Model Performance:** Many machine learning algorithms, such as gradient descent-based models, perform better when the input features are on a similar scale. Normalization ensures that no single feature dominates due to its scale.
- **Faster Convergence:** Algorithms converge faster during training when the data is normalized, reducing the time and computational resources required.
- **Reduced Sensitivity to Outliers:** Normalization can help reduce the impact of outliers, making the model more robust.
- **Consistency in Analysis:** Normalized data ensures that features contribute proportionately to the analysis, leading to more reliable and interpretable results.

Benefits to the Analysis: Normalizing the data makes the machine learning models more stable and reliable, reducing the risk of biases toward certain features due to their scale. It also ensures that the distance-based algorithms, such as k-NN and SVM, work efficiently since they rely on feature distances, which should be comparable.

Q.3 Discuss why it's essential to convert categorical variables like 'gender' into numerical representations.

Importance of Converting Categorical Variables: Categorical variables like 'gender' represent qualitative data that machine learning algorithms cannot process directly. Most algorithms require numerical inputs to perform mathematical computations. Therefore, converting categorical variables into numerical representations is essential for several reasons:

- **Compatibility with Algorithms:** Algorithms like linear regression, SVM, and neural networks require numerical inputs to perform calculations. Categorical data needs to be encoded into numbers to be used in these models.
- **Preserving Information:** Proper encoding techniques, like one-hot encoding, ensure that the categorical data is represented in a way that preserves its inherent information without introducing any ordinal relationships unless they are meaningful.
- **Improved Model Interpretability:** Converting categorical variables into numbers helps in analyzing feature importance and understanding how different categories contribute to the predictions.

Common Methods of Conversion:

- **Label Encoding:** Assigns a unique number to each category. It's simple but can introduce ordinal relationships where they don't exist.
- **One-Hot Encoding:** Creates binary columns for each category, ensuring no ordinal relationship is introduced. This method is preferred when there is no inherent order in the categories.

By converting categorical variables into numerical forms, we ensure that the machine learning models can utilize all available information effectively, leading to better predictions and more accurate analyses.